A PROJECT REPORT ON

WANDERLUST

A DevOps-Integrated, Self-Healing, and Observable Cloud Travel Booking Platform

End-to-End DevOps & Cloud Architecture using
AWS, Docker, Terraform, Jenkins, Kubernetes (EKS), Helm, Prometheus and Grafana

Submitted in partial fulfilment of the requirements of the
Bachelor of Technology in Computer Science and Engineering

Submitted by	Vecha Laxmi Swaraj Babu
Roll / Registration No.	[B42 /12307477]
Programme	B.Tech, Computer Science and Engineering
Institution	Lovely Professional University
Project Guide / Reviewer	Ajay Kumar Badhan
Academic Year	2026 - 2027
Date of Submission	October 2026
GitHub URL	https://github.com/Swarajbabu/Wanderlust-A-DevOps-Integrated-Travel-Booking-Platform

Technology Stack: AWS (VPC, EC2, ECR, EKS, ELB), Docker, Docker Compose, Terraform, Jenkins, Kubernetes, Helm, Prometheus, Grafana, Node.js, MongoDB Atlas

ABSTRACT

Modern web applications are expected to be available around the clock, to scale with demand, to recover from failures without human intervention and to be updated many times a day without interrupting users. Traditional deployment, in which a developer manually copies code to a single server, cannot meet these expectations. This project, **Wanderlust**, demonstrates how a full-stack travel and vacation-rental booking application written in Node.js, Express and MongoDB can be transformed into a production-grade, cloud-native platform by applying a complete DevOps lifecycle.

The project integrates ten tools. **Git and GitHub** provide version control and webhook events. **Docker** packages the application into a small *node:20-alpine* image using layer-cache-friendly instructions, and **Docker Compose** provides local parity with the cloud environment. **Terraform** declaratively provisions the entire **AWS** footprint in the *ap-south-1* region: a VPC with two public subnets in separate availability zones, an Internet Gateway, a Jenkins server on EC2, an Elastic Container Registry with a lifecycle policy, and an Amazon EKS cluster running Kubernetes 1.30 with a managed node group. A **Jenkins** declarative pipeline, triggered automatically by a GitHub webhook, checks out the code, builds a dual-tagged Docker image, pushes it to ECR and performs a zero-downtime rolling update on EKS.

On Kubernetes the application runs as three replicas with a rolling-update strategy of *maxSurge 1* and *maxUnavailable 0*, liveness and readiness probes, explicit CPU and memory requests and limits, a Horizontal Pod Autoscaler scaling from three to ten pods at 70 percent CPU, and Kubernetes Secrets for credentials. External traffic enters through an AWS Classic Load Balancer. **Helm** installs the *kube-prometheus-stack*, where **Prometheus** collects metrics and **Grafana** visualises them in a custom dashboard that follows Google's Golden Signals for the application and the USE method for the hosts.

In the verified deployment, Jenkins Build #12 completed all stages in 1 minute 35 seconds, and the application was served live through the AWS load balancer across three pods. The report documents the architecture, implementation, verification results, security and cost considerations, limitations and future scope of the work.

Keywords: DevOps, CI/CD, Infrastructure as Code, Docker, Terraform, Jenkins, Kubernetes, Amazon EKS, Helm, Prometheus, Grafana, Observability, Site Reliability Engineering.

TABLE OF CONTENTS

Chapter 1: INTRODUCTION & PROJECT OBJECTIVES 7

1.1 Executive Summary 7

1.2 Problem Statement & Industry Motivation 7

1.3 Objectives of the DevOps Transformation 8

1.4 The 10-Tool DevOps Stack Summary 8

1.5 Technology Stack Specification 10

1.6 Organisation of the Report 10

Chapter 2: ARCHITECTURE & HIGH-LEVEL SYSTEM DESIGN 11

2.1 End-to-End Dev-to-Cloud Workflow 11

2.2 Cloud Networking Architecture (AWS VPC, Subnets, Multi-AZ) 11

2.3 Microservices Orchestration Topology (AWS EKS) 12

2.4 Architecture Diagrams 13

Chapter 3: APPLICATION CONTAINERIZATION & LOCAL DEVELOPMENT . . . 15

3.1 Application Technology Stack 15

3.2 Docker Strategy & Multi-Stage Layer Caching 15

3.3 Security Hardening & .dockerignore Configuration 16

3.4 Local Multi-Container Parity with Docker Compose 16

Chapter 4: INFRASTRUCTURE AS CODE (IaC) USING TERRAFORM 18

4.1 Overview of Declarative Infrastructure Provisioning 18

4.2 AWS VPC & Multi-AZ Subnet Provisioning (vpc.tf) 18

4.3 Jenkins Automation Server & Dynamic Key Pairs (ec2.tf) 19

4.4 Elastic Container Registry (ECR) & Lifecycle Policies (ecr.tf) 19

4.5 Amazon EKS Cluster & Managed Node Group Creation (eks.tf) 19

4.6 IAM Roles & Policy Mappings 19

Chapter 5: CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY (CI/CD) . . 21

5.1 Jenkins Master Architecture & GitHub Webhook Triggers 21

5.2 Declarative Jenkinsfile Pipeline Stages 21

5.2.1 SCM Checkout 21

5.2.2 Docker Build & Dual-Tagging Strategy 21

5.2.3 AWS IAM Authentication & Push to ECR 22

5.2.4 Zero-Downtime Deployment to AWS EKS 22

5.2.5 Post-Build Actions & Host Storage Cleanup 22

5.3 Pipeline Execution Results 22

Chapter 6: CONTAINER ORCHESTRATION WITH KUBERNETES (AWS EKS) . . . 24

6.1 Cluster Configuration & Worker Node Topology 24

6.2 High Availability Pod Deployment (3 Replicas across Multi-AZ) 24

6.3 Zero-Downtime Rolling Update Strategy (maxSurge & maxUnavailable) 24

6.4 Container Health Probes (Liveness & Readiness Probes) 25

6.5 External Traffic Management (AWS Classic Load Balancer) 26

6.6 Horizontal Pod Autoscaling (HPA) Policy 26

6.7 Secrets Management (Kubernetes Secrets) 26

Chapter 7: CLUSTER MONITORING & SRE OBSERVABILITY 27

7.1 Monitoring Strategy: Golden Signals vs. USE Method 27

7.2 Package Management via Helm (kube-prometheus-stack) 27

7.3 Metrics Collection Architecture (Prometheus Operator & Node Exporter) 27

7.4 Grafana Dashboard Architecture & PromQL Metrics 28

7.4.1 Application Workload Tier (Healthy Pods, Restarts, RAM) 28

7.4.2 Compute Resource Tier (Per-Pod CPU Millicores & Working Set) 28

7.4.3 Host Infrastructure Tier (EC2 CPU, RAM & Disk Gauges) 28

7.5 Grafana Cluster Overview Dashboard 30

Chapter 8: SYSTEM VERIFICATION & RESULTS 31

8.1 Live Application Functional Testing 31

8.2 Live Website on AWS ELB 32

8.3 Live Booking & Checkout Flow 32

8.4 Pipeline Speed & Deployment Lead Time Results 33

Chapter 9: SECURITY, COST MANAGEMENT & TEARDOWN 34

9.1 Cloud Security Practices (IAM Least Privilege, Kubernetes Secrets, Helmet CSP) 34

9.2 AWS Cost Optimization & Automated Teardown Strategy (Terraform Destroy) 34

Chapter 10: CONCLUSION & FUTURE SCOPE 36

10.1 Key Learnings & Project Achievements 36

10.2 Limitations 36

10.3 Future Scope (GitOps with ArgoCD, Service Mesh with Istio) 37

Appendix and References 38

Appendix A: Kubernetes Command Reference Table 38

Appendix B: Git Repository & Live Links 38

References 39

LIST OF FIGURES

Figure 2.1: Dev-to-Cloud delivery flow and Jenkins stages 11

Figure 2.2: End-to-End Cloud DevOps Architecture 13

Figure 2.3: Network and runtime topology inside the VPC 14

Figure 5.1: Jenkins Build #12 pipeline stage view 23

Figure 7.1: Grafana Cluster Overview Dashboard 30

Figure 8.1: Live application homepage on the AWS ELB 32

Figure 8.2: Live booking calendar and checkout flow 32

CHAPTER 1

INTRODUCTION & PROJECT OBJECTIVES

1.1 Executive Summary

Wanderlust is a full-stack travel and vacation-rental booking application. Users can browse property listings, view locations on interactive maps, create accounts, make bookings through a checkout flow and leave reviews. The application layer is written in Node.js with the Express framework, renders pages using EJS templates, and stores its data in MongoDB Atlas, a managed cloud document database.

The focus of this project is not the application features alone but the engineering system that builds, ships, runs and observes the application. The project re-architects Wanderlust from a manually started process on a developer machine into an enterprise-style, cloud-native platform on Amazon Web Services. Every stage of the journey from *git push* to a running, monitored pod in production is automated and reproducible.

The final system is **self-healing** because Kubernetes restarts failed containers and replaces unhealthy pods, **zero-downtime** because updates are rolled out gradually behind health checks, **elastic** because a Horizontal Pod Autoscaler adjusts the replica count to load, and **observable** because Prometheus and Grafana expose the health of both the application and the underlying hosts in real time.

1.2 Problem Statement & Industry Motivation

Many student and small-team projects are deployed by logging into a server, pulling the latest code and restarting a process by hand. This approach has well-known weaknesses that DevOps practices are designed to remove:

- **Manual, error-prone releases.** Each deployment depends on the memory and discipline of a person. A skipped step or a mistyped command can take the site down.
- **Environment drift.** The application behaves differently on a laptop, a test machine and a server because the operating system, runtime versions and configuration differ.
- **Downtime during updates.** Stopping the old version before starting the new one interrupts users, and a faulty release has no automatic rollback.
- **No resilience.** A single process on a single machine is a single point of failure. A crash or an out-of-memory event stays unnoticed until users complain.
- **Unrepeatable infrastructure.** Servers created by clicking in a console cannot be recreated reliably, reviewed in version control or destroyed cleanly to control cost.
- **No visibility.** Without metrics, teams cannot see saturation, memory leaks or restarts until the system has already failed.

Industry addresses these problems with a DevOps toolchain: version control, containers, infrastructure as code, continuous integration and delivery, container orchestration and

monitoring. This project applies that toolchain end to end to a real application so that every problem listed above has a concrete, working solution.

1.3 Objectives of the DevOps Transformation

1 Containerise the application with Docker using a small base image and an efficient layer-caching strategy so that builds are fast and consistent.

2 Provide local development parity with the cloud environment using Docker Compose.

3 Provision all AWS infrastructure (network, compute, registry and Kubernetes cluster) declaratively with Terraform so it can be created and destroyed with a single command.

4 Automate the build, test-ready packaging, publishing and deployment of every commit through a Jenkins declarative pipeline triggered by a GitHub webhook.

5 Run the application on Amazon EKS with high availability across two availability zones, zero-downtime rolling updates, health probes, resource governance and horizontal autoscaling.

6 Manage configuration and credentials outside the image using Kubernetes Secrets and environment variables.

7 Install and operate a production-style monitoring stack with Helm and build a Grafana dashboard that follows the Golden Signals and USE methods.

8 Verify the system functionally on the live cloud endpoint and document deployment lead time, health and behaviour.

9 Control cloud cost through lifecycle policies, right-sized instances and a documented teardown procedure.

1.4 The 10-Tool DevOps Stack Summary

The project covers the complete DevOps lifecycle using the following ten tools and platforms. Table 1 summarises the role each one plays; later chapters describe each in detail.

Table 1: The ten-tool DevOps stack used in Wanderlust

No.	Tool	Lifecycle Phase	Role in the Project
1	Git & GitHub	Source control	Version control, branching and webhook emission on every push
2	Docker	Packaging	Multi-stage-style, layer-cached image on node:20-alpine
3	Docker Compose	Local development	Local runtime parity with the cloud; connects to MongoDB Atlas
4	Terraform	Infrastructure as Code	Declarative provisioning of VPC, EC2, ECR and EKS
5	AWS	Cloud platform	VPC, subnets, IGW, EC2, ECR, EKS and Elastic Load Balancer
6	Jenkins	CI/CD	Declarative pipeline: checkout, build, push, deploy, cleanup
7	Kubernetes (EKS)	Orchestration	Pods, self-healing, rolling updates, HPA, services, secrets
8	Helm	Package management	Installs kube-prometheus-stack into the cluster
9	Prometheus	Monitoring	Pull-based time-series metrics from pods, nodes and the cluster
10	Grafana	Visualisation / SRE	Dashboards for Golden Signals and USE-method host metrics

1.5 Technology Stack Specification

Table 2: Technology stack specification

Category	Tool / Service	Version / Flavor	Role in Project
Source Control	Git & GitHub	v2.34+	Distributed version control and webhook emission
Container Engine	Docker	Engine v24+ (Alpine 3.19)	Image build and layer caching
Local Orchestration	Docker Compose	v2+	Local development environment parity
Cloud Provider	AWS	Region: ap-south-1	VPC, Subnets, EC2, ECR, EKS, ELB
Infrastructure as Code	Terraform	v1.5+ (HashiCorp)	Declarative provisioning of AWS infrastructure
CI/CD Automation	Jenkins	v2.580+ (Java 21)	Declarative pipeline; ECR and EKS deployments
Container Orchestration	Kubernetes (AWS EKS)	v1.30	Pod lifecycle, self-healing, rolling updates, HPA
Package Manager	Helm	v3+ / v4+	Packaging of kube-prometheus-stack
Metrics Engine	Prometheus	Prometheus Operator	Time-series metrics via pull model
Visualisation / SRE	Grafana	v11+	Dashboards for Golden Signals and USE method
Backend / Database	Node.js / MongoDB Atlas	Node v20 / MongoDB v7	REST API and document database

1.6 Organisation of the Report

Chapter 2 presents the overall architecture. Chapter 3 describes containerisation and local development. Chapter 4 covers the Terraform infrastructure. Chapter 5 explains the Jenkins CI/CD pipeline. Chapter 6 details Kubernetes orchestration on EKS. Chapter 7 describes monitoring and observability. Chapter 8 presents verification and results. Chapter 9 discusses security, cost and teardown. Chapter 10 concludes with learnings, limitations and future scope. The appendix lists a command reference, repository links and references.

CHAPTER 2

ARCHITECTURE & HIGH-LEVEL SYSTEM DESIGN

2.1 End-to-End Dev-to-Cloud Workflow

The platform follows a GitOps-style flow in which the Git repository is the single source of truth. A change moves from a developer's machine to production through the following automated sequence:

1 The developer commits and pushes code to the **main** branch of the GitHub repository.

2 GitHub sends an HTTP POST webhook to the Jenkins server's */github-webhook/* endpoint within a couple of seconds of the push.

3 Jenkins, running on an AWS EC2 instance, starts the declarative pipeline defined in the repository's Jenkinsfile.

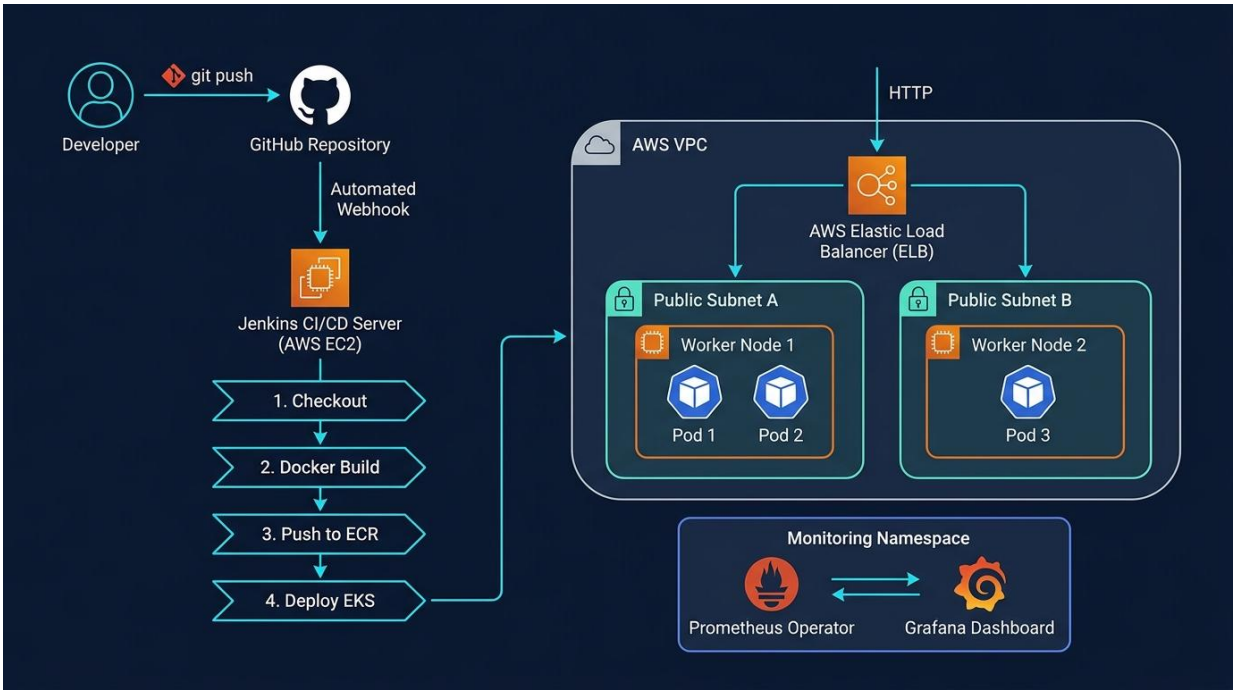
4 The pipeline checks out the exact commit, builds the Docker image and tags it with the build number and *latest*.

5 Jenkins authenticates to Amazon ECR using temporary IAM credentials and pushes both tags to the *wanderlust-app* repository.

6 Jenkins runs *kubectl set image* against the EKS cluster. Kubernetes performs a rolling update and Jenkins waits for *kubectl rollout status* to confirm success.

7 The AWS Classic Load Balancer continues to route user traffic to healthy pods throughout the update.

8 Prometheus scrapes metrics from the cluster continuously and Grafana displays the resulting health and resource dashboards.



2.2 Cloud Networking Architecture (AWS VPC, Subnets, Multi-AZ)

All resources live inside a dedicated Amazon Virtual Private Cloud so that the project is isolated from other workloads in the AWS account. The network is defined entirely in Terraform (*vpc.tf*) and is summarised in Table 3.

Table 3: Network design parameters

Component	Configuration	Purpose
Region	ap-south-1 (Mumbai)	Low latency for users in India
VPC	CIDR 10.0.0.0/16	Isolated private address space of 65,536 addresses
Public Subnet A	10.0.1.0/24 in ap-south-1a	Hosts a worker node and pods in availability zone A
Public Subnet B	10.0.2.0/24 in ap-south-1b	Hosts a worker node and pods in availability zone B
Internet Gateway	Attached to the VPC	Provides internet access to public subnets
Route Table	Default route 0.0.0.0/0 to the IGW	Routes outbound and inbound internet traffic
Subnet tag	kubernetes.io/role/elb = 1	Lets Kubernetes automatically place public load balancers

Using two subnets in two availability zones is the foundation of high availability. If one availability zone suffers an outage, the worker node and pods in the other zone continue to serve traffic, and the EKS control plane, which AWS manages across multiple zones, remains reachable. The subnet tag *kubernetes.io/role/elb* is a Kubernetes-on-AWS convention: when a Service of type LoadBalancer is created, the cloud controller looks for subnets with this tag and attaches the load balancer there without manual configuration.

2.3 Microservices Orchestration Topology (AWS EKS)

Compute is provided by Amazon Elastic Kubernetes Service. AWS operates the Kubernetes control plane (API server, scheduler, controller manager and etcd), while the project operates a **managed node group** of two *t3.small* instances running Amazon Linux 2023. The Wanderlust Deployment runs three pod replicas which the scheduler spreads across the two worker nodes, so the loss of one node leaves at least one pod running while Kubernetes reschedules the others.

A separate Jenkins server runs outside the cluster on a *t3.micro* EC2 instance with Ubuntu 22.04 LTS. Keeping the CI server outside the cluster it deploys to avoids a circular dependency: if the cluster were unhealthy, the pipeline that could repair it would still be available.

Table 4: Principal deployed resources

Resource	Name / Value	Notes
EKS Cluster	wanderlust-cluster	Kubernetes v1.30
Node Group	2 x t3.small	Amazon Linux 2023, managed node group
Container Registry	wanderlust-app (Amazon ECR)	Lifecycle policy for untagged images
CI Server	EC2 t3.micro, Ubuntu 22.04	Jenkins; ports 22, 8080, 80 open via security group
Load Balancer	AWS Classic ELB	Created by Kubernetes Service type LoadBalancer
Application Pods	3 replicas (HPA range 3 to 10)	Container port 8081
Monitoring Namespace	monitoring	Prometheus Operator, Node Exporters, Grafana

2.4 Architecture Diagrams

Figure 2.2 is the hero architecture diagram of the project and shows the full flow from the Wanderlust repository through Docker, Jenkins, Helm and Kubernetes on AWS to the monitoring dashboards. Figure 2.3 shows the network and runtime topology inside the VPC.

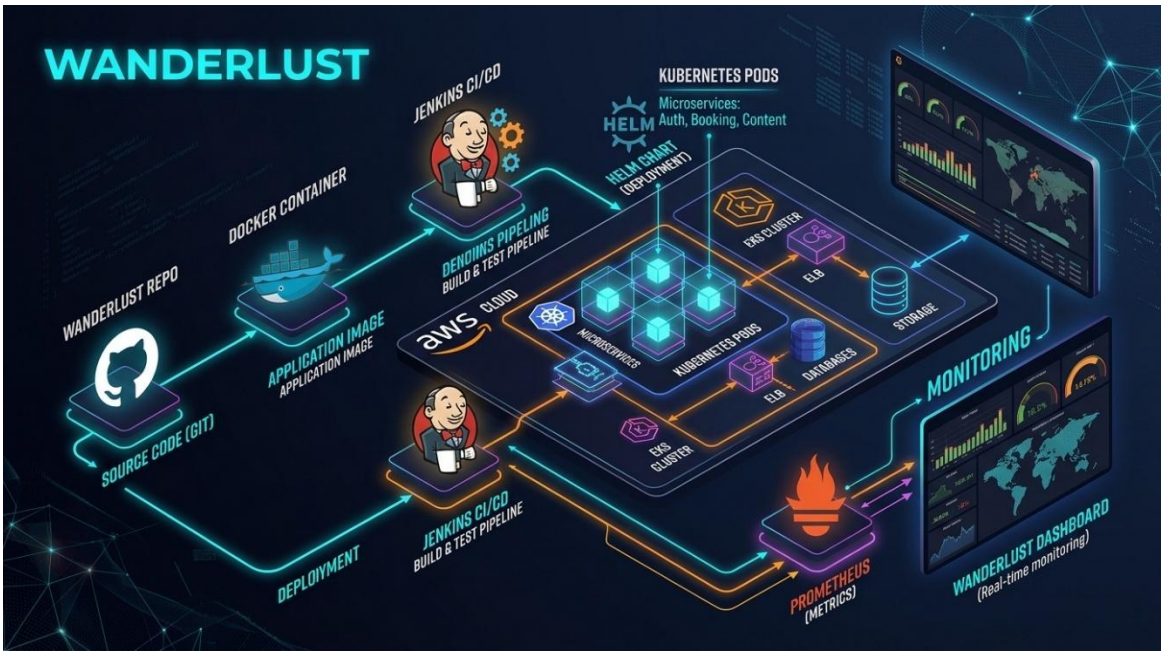


Figure 2.2: End-to-End Cloud DevOps Architecture Diagram illustrating the GitOps flow from GitHub to AWS EKS via Jenkins, Prometheus, and Grafana.

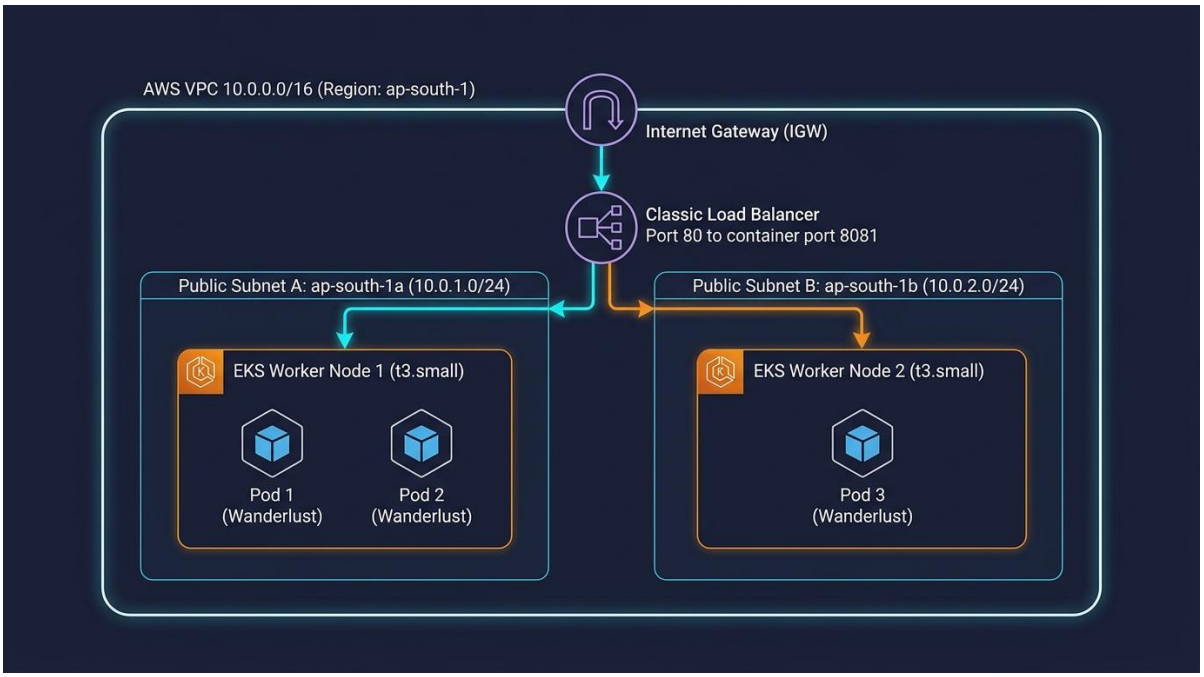


Figure 2.3: Network and runtime topology: VPC, two public subnets across availability zones, Classic Load Balancer and three application pods on two EKS worker nodes.

CHAPTER 3

APPLICATION CONTAINERIZATION & LOCAL DEVELOPMENT

3.1 Application Technology Stack

The Wanderlust application is a server-rendered web application. Its major components are:

- **Node.js 20** as the JavaScript runtime, chosen for its non-blocking I/O model, which suits an application that mostly waits on database and network operations.
- **Express.js** as the web framework, providing routing, middleware and request handling. The repository separates concerns into *routes*, *controllers*, *modals* (data models), *middleware.js* and a *schema.js* validation layer.
- **EJS** templates in the *views* directory for server-side rendering, with static assets in *public*.
- **MongoDB Atlas** as a managed cloud database, so that the database tier needs no servers inside the Kubernetes cluster and the application pods remain stateless.
- Application features include property listings, user authentication with sessions, a booking and checkout flow, reviews, image uploads and interactive maps.

Keeping the application pods **stateless** is an important design decision. Session state and data live in MongoDB Atlas, so any pod can serve any request. This is what allows Kubernetes to add, remove and replace pods freely behind the load balancer.

3.2 Docker Strategy & Multi-Stage Layer Caching

Docker packages the application and its dependencies into an immutable image that runs identically on a laptop, the Jenkins server and the EKS worker nodes. The project's Dockerfile is deliberately short:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev --legacy-peer-deps
COPY . .
EXPOSE 8081
ENV PORT=8081
ENV NODE_ENV=production
CMD ["npm", "start"]
```

Base image. *node:20-alpine* is based on Alpine Linux, which is far smaller than a full Debian-based image. A smaller image downloads faster to worker nodes, starts faster during rolling updates and contains fewer operating-system packages, which reduces the attack surface.

Layer caching. Docker caches each instruction as a layer and reuses a layer only if the instruction and everything before it are unchanged. The Dockerfile therefore copies only *package*.json* first and runs the dependency installation, and only afterwards copies the rest of the source code.

Because application code changes on almost every commit while dependencies change rarely, the expensive *npm ci* layer is served from cache on most builds. Only when *package.json* or the lock file changes is the dependency layer rebuilt.

Reproducible installs. *npm ci* installs exactly the versions recorded in *package-lock.json* and fails if the lock file and manifest disagree, which makes builds deterministic. The *--omit=dev* flag leaves out development dependencies, keeping the production image lean.

Runtime configuration. The container listens on port 8081 and runs in production mode. Secrets such as database credentials are never baked into the image; they are injected at runtime as environment variables, from a Kubernetes Secret in the cluster and from an environment file locally.

3.3 Security Hardening & .dockerignore Configuration

The *.dockerignore* file excludes files that must not enter the build context or the image. Excluding the *.git* history, test directories, Terraform files and local environment files has three benefits: the build context sent to the Docker daemon is smaller, so builds are faster; the final image is smaller; and sensitive or irrelevant material, such as infrastructure code and local secrets, can never be leaked through an image pushed to a registry.

Table 5: Image hardening and hygiene measures

Measure	Benefit
Alpine-based minimal base image	Smaller attack surface and faster pulls
Dependencies installed with npm ci --omit=dev	Deterministic builds without development tooling in production
.dockerignore excludes .git, tests, terraform and env files	Smaller context and no accidental leakage of secrets or infrastructure code
No credentials in the image	Secrets are injected at runtime from Kubernetes Secrets
Immutable build-number tags	Every image is traceable to a pipeline run and can be rolled back to

3.4 Local Multi-Container Parity with Docker Compose

Docker Compose, defined in *docker-compose.yml*, lets a developer start the containerised application locally with one command. It connects the local container to MongoDB Atlas and propagates configuration through environment variables, so the local run uses the same image, the same port and the same variable names as the cloud deployment. This is the practical meaning of **environment parity**: if the application works in Compose, the same image and configuration contract will work on Kubernetes.

```
# Build the image and start the application locally
docker compose up --build

# Run in the background and view logs
docker compose up -d
docker compose logs -f

# Stop and remove the containers
docker compose down
```

Compose is used only for development parity. Production orchestration, scaling and self-healing are the responsibility of Kubernetes, which is covered in Chapter 6.

CHAPTER 4

INFRASTRUCTURE AS CODE (IaC) USING TERRAFORM

4.1 Overview of Declarative Infrastructure Provisioning

Infrastructure as Code means describing infrastructure in text files that are versioned, reviewed and executed by a tool rather than creating resources manually in a console. Terraform uses a declarative language, HCL, in which the engineer states the desired end state, and Terraform computes and applies the changes needed to reach it. The configuration for this project is organised into small files inside the *terraform/* directory, each responsible for one concern:

Table 6: Terraform file organisation

File	Responsibility
`provider.tf`	Configures the AWS provider pinned to region ap-south-1
`vpc.tf`	VPC, two public subnets, Internet Gateway and route tables
`ec2.tf`	Jenkins EC2 instance, generated key pair and security group
`ecr.tf`	Elastic Container Registry and lifecycle policy
`eks.tf`	EKS cluster, managed node group and IAM roles

The standard Terraform workflow is *init* to download providers, *plan* to preview changes and *apply* to create resources. Because the entire environment is described in code, it is reproducible, reviewable through Git, and removable with a single *terraform destroy*, which is essential for controlling cost in a learning environment.

```
cd terraform
terraform init
terraform apply -auto-approve
```

4.2 AWS VPC & Multi-AZ Subnet Provisioning (vpc.tf)

The file *vpc.tf* creates a VPC with the CIDR block *10.0.0.0/16* and two public subnets, *10.0.1.0/24* in *ap-south-1a* and *10.0.2.0/24* in *ap-south-1b*. An Internet Gateway is attached to the VPC and a route table sends *0.0.0.0/0* traffic to the gateway, with an association to each subnet. The subnets carry the tag *kubernetes.io/role/elb = 1*, which allows the Kubernetes cloud controller to discover them and place the public load balancer automatically.

Two subnets in two availability zones are the minimum that EKS requires for its control-plane network interfaces and are also the minimum for a meaningful high-availability design.

4.3 Jenkins Automation Server & Dynamic Key Pairs (ec2.tf)

The file *ec2.tf* provisions the CI server: an Ubuntu 22.04 LTS instance of type *t3.micro*. Rather than creating an SSH key pair by hand, Terraform generates a **4096-bit RSA private key** using the TLS provider, registers the public half as an AWS key pair and writes the private key to a local file (*jenkins_key.pem*). This removes a manual step and makes the key part of the reproducible workflow.

A security group attached to the instance opens three ports: **22** for SSH administration, **8080** for the Jenkins web interface and webhook endpoint, and **80** for HTTP. Port 8080 must be reachable from the internet so that GitHub can deliver webhooks.

4.4 Elastic Container Registry (ECR) & Lifecycle Policies (ecr.tf)

The file *ecr.tf* creates a private ECR repository named *wanderlust-app* that stores the application images built by Jenkins. Because every pipeline run pushes a new tagged image, storage would grow without bound. A **lifecycle policy** therefore expires untagged images older than 30 days. This automatic housekeeping controls storage cost without any manual clean-up and without touching the build-number tagged images that are needed for rollbacks.

4.5 Amazon EKS Cluster & Managed Node Group Creation (eks.tf)

The file *eks.tf* creates the Kubernetes platform: an EKS cluster named *wanderlust-cluster* running Kubernetes version 1.30 in the two public subnets, and a managed node group of two *t3.small* instances running Amazon Linux 2023. A managed node group means AWS handles node provisioning, registration with the cluster and lifecycle operations such as draining during updates, which removes a large amount of operational work compared with self-managed nodes.

Once the cluster exists, the local *kubectl* client is connected to it with the AWS CLI, which writes the cluster endpoint and authentication settings into the kubeconfig file:

```
aws eks update-kubeconfig --region ap-south-1 --name wanderlust-cluster
kubectl get nodes
```

4.6 IAM Roles & Policy Mappings

AWS Identity and Access Management controls what each component is allowed to do. The EKS control plane and the worker nodes each assume a dedicated IAM role with AWS-managed policies attached. The mapping is shown in Table 7.

Table 7: IAM policies attached to the EKS roles

Policy	Attached To	Purpose
`AmazonEKSClusterPolicy`	Cluster role	Allows the EKS control plane to manage AWS resources such as network interfaces and load balancers
`AmazonEKSWorkerNodePolicy`	Node role	Allows worker nodes to connect to the EKS cluster
`AmazonEKS_CNI_Policy`	Node role	Allows the VPC CNI plugin to assign pod IP addresses from the VPC
`AmazonEC2ContainerRegistryReadOnly`	Node role	Allows nodes to pull images from ECR but not to push or delete them

Separating roles in this way follows the principle of least privilege: worker nodes can only *pull* images, while the permission to *push* images belongs to the Jenkins pipeline, which authenticates with temporary credentials injected at build time.

CHAPTER 5

CONTINUOUS INTEGRATION & CONTINUOUS DELIVERY (CI/CD)

5.1 Jenkins Master Architecture & GitHub Webhook Triggers

Jenkins is an open-source automation server. In this project a single Jenkins controller runs on the Terraform-provisioned EC2 instance (Java 21, Jenkins 2.580 or later) and executes the pipeline directly on that host, which also has Docker, the AWS CLI and kubectl installed so that it can build images, push to ECR and talk to the EKS API.

Builds are started by a **GitHub webhook** rather than by polling. In the repository settings a webhook points to *http://<JENKINS_EC2_IP>:8080/github-webhook/*. On every *git push* GitHub sends a JSON payload to this URL, and Jenkins begins the pipeline within about two seconds. Push-based triggering is faster and wastes no resources compared with periodic polling of the repository.

The pipeline is defined as code in the *Jenkinsfile* at the root of the repository. Keeping the pipeline in version control means changes to the delivery process are reviewed and tracked in the same way as application code.

5.2 Declarative Jenkinsfile Pipeline Stages

The declarative pipeline has five stages, executed in order. A failure in any stage stops the pipeline and prevents later stages from running, so a broken build can never reach the cluster.

5.2.1 SCM Checkout

Jenkins clones the repository at the exact Git commit that triggered the webhook. Building the precise commit, rather than the latest state of a branch, guarantees that the image corresponds to a known version of the source code even if further commits are pushed while the build runs.

5.2.2 Docker Build & Dual-Tagging Strategy

Jenkins builds the Docker image and gives it two tags: the Jenkins **build number** and **latest**. The build-number tag is immutable and unique; it identifies precisely which pipeline run produced the image and provides a stable target for rollbacks. The *latest* tag is a convenient moving pointer to the most recent successful build.

```
# Representative commands for the build stage
docker build -t wanderlust-app:${BUILD_NUMBER} -t wanderlust-app:latest .
```

5.2.3 AWS IAM Authentication & Push to ECR

Jenkins authenticates to ECR using short-lived tokens obtained with AWS credentials injected by the *AmazonWebServicesCredentialsBinding* plugin. The credentials are stored in the Jenkins credential store and exposed to the build only as masked environment variables, so they never appear in the Jenkinsfile or in the console log. After login the pipeline tags the image with the ECR repository path and pushes both tags.

```
# Representative commands for the push stage
aws ecr get-login-password --region ap-south-1 \
  | docker login --username AWS --password-stdin <account-id>.dkr.ecr.ap-south-
1.amazonaws.com docker push <account-id>.dkr.ecr.ap-south-1.amazonaws.com/wanderlust-
app:${BUILD_NUMBER} docker push <account-id>.dkr.ecr.ap-south-
1.amazonaws.com/wanderlust-app:latest
```

5.2.4 Zero-Downtime Deployment to AWS EKS

The deploy stage updates the running Deployment to the new image with *kubectl set image*. This changes the pod template, and Kubernetes begins a rolling update, bringing up a new pod, waiting for its readiness probe to pass and only then removing an old pod. The pipeline then runs *kubectl rollout status*, which blocks until the rollout completes and returns a failure code if it does not. The pipeline therefore reports success only when the new version is actually healthy and serving traffic.

```
# Representative commands for the deploy stage
kubectl set image deployment/wanderlust-deployment \
  wanderlust=<account-id>.dkr.ecr.ap-south-1.amazonaws.com/wanderlust-app:${BUILD_NUMBER}
kubectl rollout status deployment/wanderlust-deployment
```

5.2.5 Post-Build Actions & Host Storage Cleanup

Each build leaves layers and dangling images on the Jenkins host. A small *t3.micro* instance has limited disk, so the post-build section prunes dangling Docker images to prevent the disk from filling up and breaking future builds. This runs regardless of whether the build passed or failed.

5.3 Pipeline Execution Results

Figure 5.1 shows the Jenkins stage view for Build #12. All five stages completed successfully and the whole run, from webhook trigger to a verified rollout on EKS, took **1 minute 35 seconds**.

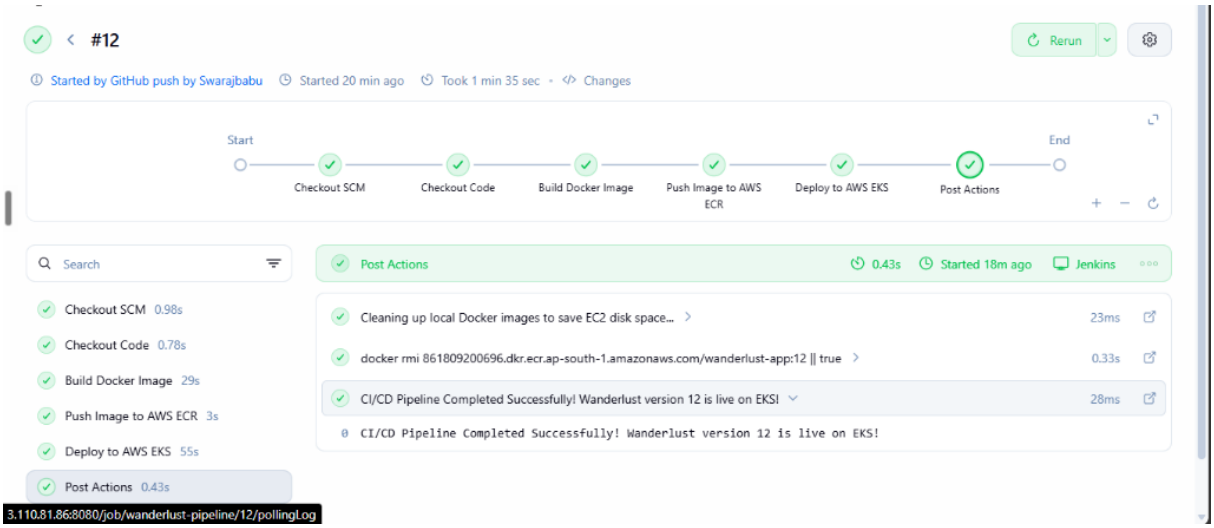


Figure 5.1: Automated Jenkins Declarative Pipeline (Build #12) demonstrating successful end-to-end execution of all 5 stages in 1m 35s.

Table 8: CI/CD pipeline stage summary

Stage	Action	Outcome / Guarantee
1. SCM Checkout	Clone triggering commit	Build matches a known source revision
2. Build Docker Image	Build and tag with build number and latest	Immutable, traceable artefact
3. Push to ECR	Authenticate with IAM and push	Image available to the cluster
4. Deploy to EKS	kubectl set image + rollout status	Zero-downtime, verified release
5. Post Actions	Prune dangling images	Host disk space preserved

A commit-to-production lead time of under two minutes, with no manual step, is the central achievement of the CI/CD design. It makes small, frequent releases practical and reduces the risk of each release.

CHAPTER 6

CONTAINER ORCHESTRATION WITH KUBERNETES (AWS EKS)

6.1 Cluster Configuration & Worker Node Topology

Kubernetes is the system that keeps the application in its desired state. The engineer declares what should be running, three replicas of the Wanderlust container for instance, and Kubernetes continuously compares reality with that declaration and corrects any difference. All manifests for the project reside in the *k8s/* directory and are applied with *kubectl apply*:

```
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/hpa.yaml
```

The cluster has two worker nodes. Pods are scheduled onto nodes by the Kubernetes scheduler based on available resources; the command *kubectl get pods -o wide* shows the three pods distributed across the two EC2 workers.

6.2 High Availability Pod Deployment (3 Replicas across Multi-AZ)

The Deployment in *deployment.yaml* requests three replicas. A Deployment manages a ReplicaSet, which ensures that the requested number of pods is always running. If a pod crashes, the node fails or a pod is deleted, the ReplicaSet creates a replacement automatically. With the worker nodes in different availability zones, the pods are spread so that no single zone failure can remove the whole application.

6.3 Zero-Downtime Rolling Update Strategy (maxSurge & maxUnavailable)

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

These two parameters define how the update proceeds. **maxSurge: 1** allows Kubernetes to create one extra pod above the desired count during an update, so the cluster temporarily runs four pods. **maxUnavailable: 0** forbids Kubernetes from taking any existing pod out of service until a replacement is ready. The effect is the following sequence for each pod: a new pod is created; it starts; its readiness probe passes; the Service begins routing traffic to it; only then is one old pod terminated. This repeats until all three pods run the new version, and at no moment is capacity below three ready pods.

Table 9: Rolling update walkthrough for a three-replica Deployment

Step	Old-version pods	New-version pods	Ready capacity
0. Start	3	0	3
1. Surge	3	1 (starting)	3
2. New pod ready	3	1	4
3. Remove one old	2	1	3
4. Repeat until complete	0	3	3

If the new version is faulty and its readiness probe never passes, the rollout stalls with the old pods still serving all traffic, so users are unaffected. The pipeline's *rollout status* check then fails and the build is marked as failed. An operator can return to the previous revision instantly with *kubectl rollout undo*.

6.4 Container Health Probes (Liveness & Readiness Probes)

Kubernetes cannot know whether an application is healthy unless it is told how to check. The Deployment defines two HTTP probes against the */listings* endpoint on port 8081:

- **Readiness probe** (every 5 seconds). Determines whether a pod is ready to receive traffic. A pod that fails is removed from the Service endpoints, so the load balancer does not send users to it. This is what makes rolling updates safe.
- **Liveness probe** (every 10 seconds). Determines whether the container is still working. If it fails repeatedly, for example because of a deadlock, Kubernetes restarts the container automatically. This is the self-healing behaviour.

Using */listings* as the probe target is a meaningful check because the page renders data from MongoDB; a pod that cannot reach the database fails the probe instead of silently serving errors.

Table 10: Resource requests and limits per pod

Resource	Request (guaranteed)	Limit (maximum)
CPU	100m (0.1 core)	500m (0.5 core)
Memory	128Mi	512Mi

Requests tell the scheduler how much capacity to reserve when placing the pod, while **limits** cap consumption. A container that exceeds its CPU limit is throttled; one that exceeds its memory limit is terminated with an *OOMKilled* status and restarted. Setting both prevents one misbehaving pod from starving its neighbours on a small *t3.small* node.

6.5 External Traffic Management (AWS Classic Load Balancer)

The *service.yaml* manifest declares a Service of type **LoadBalancer**, mapping public port 80 to container port 8081. When this Service is created, the Kubernetes cloud controller calls the AWS API and provisions an **AWS Classic Elastic Load Balancer** in the tagged public subnets, then registers the worker nodes as its targets. Traffic from the internet reaches the ELB on port 80 and is distributed across the nodes and on to the healthy pods in a round-robin fashion. Only pods that pass their readiness probe receive traffic.

```
kubectl get svc
# EXTERNAL-IP column shows the ELB hostname, for example:
#  a51fe5ad2863047968c94d092fc4c0ce-412112320.ap-south-1.elb.amazonaws.com
```

6.6 Horizontal Pod Autoscaling (HPA) Policy

The *hpa.yaml* manifest defines a Horizontal Pod Autoscaler that targets **70 percent average CPU utilisation** relative to the pods' CPU requests, with a minimum of 3 and a maximum of 10 replicas. When load increases and average CPU rises above 70 percent, the HPA increases the replica count, up to ten; when load drops, it scales back down to three. This gives the application elasticity without manual intervention and keeps the baseline high-availability footprint at three pods.

The HPA obtains CPU measurements from the Kubernetes Metrics API, so a metrics provider such as the Metrics Server must be present in the cluster for autoscaling decisions to work. Manual scaling remains available at any time, for example *kubectl scale deployment wanderlust-deployment --replicas=5*.

6.7 Secrets Management (Kubernetes Secrets)

The database connection string and the cookie/session secret are stored in a Kubernetes Secret defined in *secret.yaml* and are supplied to the pods as environment variables. This keeps credentials out of the Docker image and out of the Deployment manifest. The values in a Secret manifest are Base64-encoded, which is an encoding and not encryption; the protection therefore relies on restricting access to the cluster and the manifest, and on keeping the real secret file out of public version control. Section 10.3 discusses stronger options such as AWS Secrets Manager.

CHAPTER 7

CLUSTER MONITORING & SRE OBSERVABILITY

7.1 Monitoring Strategy: Golden Signals vs. USE Method

A system that cannot be observed cannot be operated reliably. The monitoring design in this project combines two well-known frameworks, each applied to the layer where it is most useful.

Google's Four Golden Signals (latency, traffic, errors and saturation) are designed for user-facing services. In the application tier, the dashboard tracks the Kubernetes-level signals that reflect service health: the number of healthy replicas, pod restart counts and memory and CPU consumption against configured limits.

Brendan Gregg's USE Method examines every resource in terms of **U**tilisation, **S**aturation and **E**rrors, and is designed for infrastructure. In the host tier, the dashboard shows CPU, memory and disk utilisation of the EC2 worker nodes, which shows how close each machine is to its limits.

Table 11: Mapping of monitoring methods to dashboard tiers

Method	Applied To	Example Indicators
Golden Signals	Application workload	Healthy replicas, pod restarts, pod CPU and memory, network throughput
USE Method	Host infrastructure	Node CPU %, node memory %, node disk %, ready nodes

7.2 Package Management via Helm (kube-prometheus-stack)

Helm is the package manager for Kubernetes. A Helm *chart* bundles all the manifests needed for a complex application, and a *release* is one installed instance of it. Installing a full monitoring stack by hand would require dozens of manifests; with Helm it is one command using the community *kube-prometheus-stack* chart, which installs the Prometheus Operator, Prometheus, Alertmanager, kube-state-metrics, Node Exporter and Grafana together with default dashboards.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

The stack is installed in its own *monitoring* namespace, which separates operational tooling from the application namespace and makes it easy to remove cleanly during teardown.

7.3 Metrics Collection Architecture (Prometheus Operator & Node Exporter)

Prometheus uses a **pull model**: it periodically scrapes HTTP metrics endpoints and stores the results in a time-series database. In this deployment the scrape interval is 15 seconds. Three sources feed the dashboard:

- **cAdvisor / kubelet** exposes container-level metrics such as *container_cpu_usage_seconds_total* and *container_memory_working_set_bytes*.
- **kube-state-metrics** converts Kubernetes object state into metrics such as available replicas, container restart counts and node conditions.
- **Node Exporter**, deployed as a DaemonSet so that one copy runs on every worker node, exposes host-level CPU, memory, disk and network metrics.

The **Prometheus Operator** manages Prometheus declaratively through custom resources such as ServiceMonitor, so the set of scraped targets updates automatically as pods and nodes come and go. Grafana is configured with Prometheus as its data source and queries it using PromQL.

7.4 Grafana Dashboard Architecture & PromQL Metrics

A custom dashboard, exported as *k8s/wanderlust-grafana-dashboard.json*, is imported into Grafana through Dashboards, New, Import. Access during development is by port-forwarding, which avoids exposing Grafana publicly:

```
kubectl port-forward svc/prometheus-stack-grafana 3000:80 -n monitoring
# Open http://localhost:3000 in a browser
```

The dashboard consists of ten panels in three tiers. Table 12 lists every panel with its query and the reason it is monitored.

7.4.1 Application Workload Tier (Healthy Pods, Restarts, RAM)

This tier answers the question: *is the application healthy right now?* **Healthy Replicas** counts available replicas of the Deployment and should read three. **Pod Restarts (1h)** counts container restarts in the last hour; a non-zero value reveals crashes, infinite loops or OOMKilled terminations that Kubernetes would otherwise hide by silently restarting the container. **Total App Memory** shows the memory working set across all application pods, and **Cluster Ready Nodes** confirms that all EC2 workers report a healthy kubelet.

7.4.2 Compute Resource Tier (Per-Pod CPU Millicores & Working Set)

This tier answers: *how much of its allowance is each pod using?* **Pod CPU Usage** plots CPU consumption per pod against the 500m limit so that throttling is visible. **Pod Memory (Working Set)** plots memory per pod; a steadily rising line indicates a memory leak in the Node.js process long before the 512Mi limit triggers an OOM kill. **Network Throughput** shows inbound traffic per pod.

7.4.3 Host Infrastructure Tier (EC2 CPU, RAM & Disk Gauges)

This tier applies the USE method to the nodes. **Node CPU Utilisation**, **Node Memory Utilisation** and **Node Disk Space** are shown as gauges. High values warn of saturation: CPU saturation suggests adding nodes, memory saturation risks kernel out-of-memory events, and high disk usage can trigger kubelet *DiskPressure* evictions of pods.

Table 12: Grafana dashboard panels and PromQL expressions

Panel	PromQL Expression	Why It Is Tracked
Healthy Replicas (Stat)	`sum(kube_deployment_status_replicas_available{deployment="wanderlust-deployment"})`	Verifies all 3 pods are running and serving traffic; alert if below 3
Pod Restarts 1h (Stat)	`sum(increase(kube_pod_container_status_restarts_total{namespace="default", pod=~"wanderlust-.*"}[1h]))`	Detects crashes, infinite loops and OOMKilled terminations
Total App Memory (Stat)	`sum(container_memory_working_set_bytes{namespace="default", pod=~"wanderlust-.*", container!=""})`	Overall RAM footprint of all Node.js processes
Cluster Ready Nodes (Stat)	`sum(kube_node_status_condition{condition="Ready", status="true"})`	Confirms every EC2 worker has a healthy kubelet heartbeat
Pod CPU Usage (Time Series)	`sum(rate(container_cpu_usage_seconds_total{namespace="default", pod=~"wanderlust-.*", container!=""}[5m])) by (pod)`	CPU per pod against the 500m limit; detects throttling
Pod Memory Working Set (Time Series)	`sum(container_memory_working_set_bytes{namespace="default", pod=~"wanderlust-.*", container!=""}) by (pod)`	Finds memory leaks before the container limit is hit
Node CPU Utilisation % (Gauge)	`100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`	Host compute saturation
Node Memory Utilisation % (Gauge)	`(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100`	Host memory saturation; avoids kernel OOM
Node Disk Space % (Gauge)	`100 - ((node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"}) * 100)`	EBS usage; avoids DiskPressure evictions
Network Throughput Rx/Tx (Time Series)	`sum(rate(container_network_receive_bytes_total{namespace="default", pod=~"wanderlust-.*"}[5m])) by (pod)`	Inbound and outbound traffic per pod

7.5 Grafana Cluster Overview Dashboard

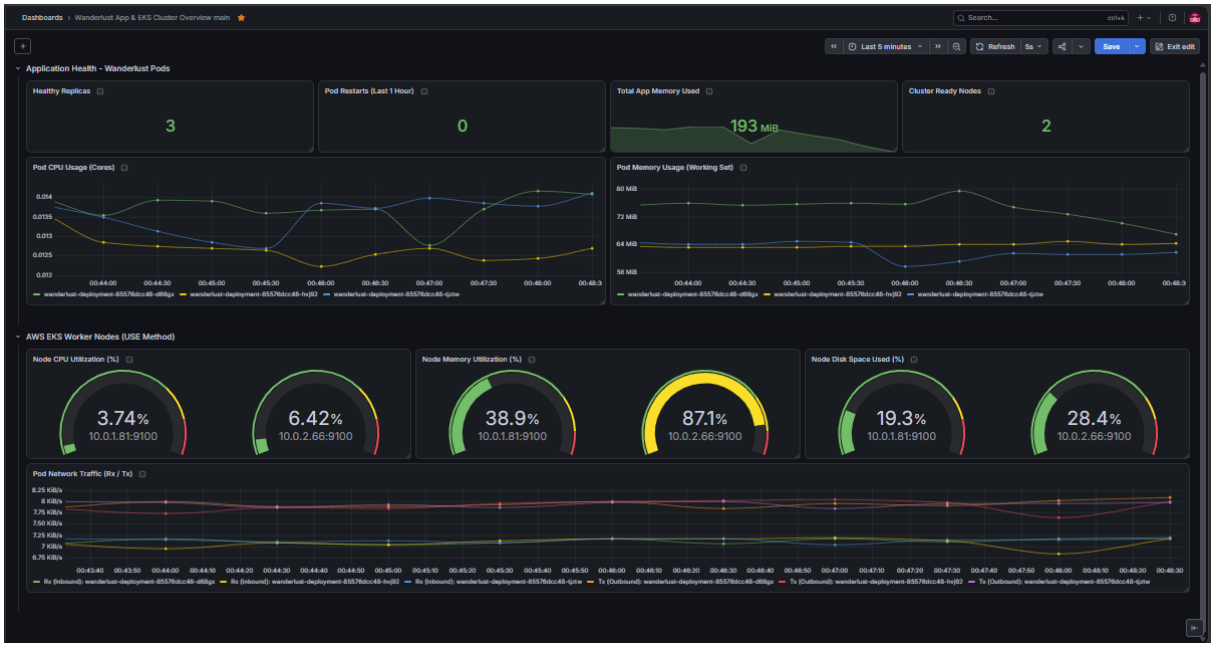


Figure 7.1: Real-time Grafana Observability Dashboard displaying Golden Signals (Pod replicas, Restarts, Memory) and EC2 Host USE metrics.

CHAPTER 8

SYSTEM VERIFICATION & RESULTS

8.1 Live Application Functional Testing

After deployment, the system was verified from the outside, as a user would experience it, and from the inside, using Kubernetes tooling. The checks were designed to confirm each claim made in the design chapters.

Table 13: Verification checklist and observed outcomes

Verification	Method	Result
Application reachable on the internet	Open the ELB hostname at /listings in a browser	Listings page served over port 80
Dynamic data from the database	Browse listings and detail pages	Data loaded from MongoDB Atlas
Session and cookie handling	Log in and navigate across pages	Session persisted across requests and pods
Booking and checkout flow	Select dates and open the checkout page	Calendar and checkout rendered correctly
High availability	kubectl get pods -o wide	3 pods distributed over 2 worker nodes
Automated delivery	Push a commit to main	Webhook triggered Jenkins; Build #12 passed
Zero-downtime rollout	kubectl rollout status during deploy	Rollout completed without service interruption
Observability	Open the Grafana dashboard	All tiers display live metrics

Because session state is held outside the pods, a user whose consecutive requests land on different pods through the load balancer keeps a continuous session. This confirms that the stateless design works as intended behind a multi-replica Service.

8.2 Live Website on AWS ELB

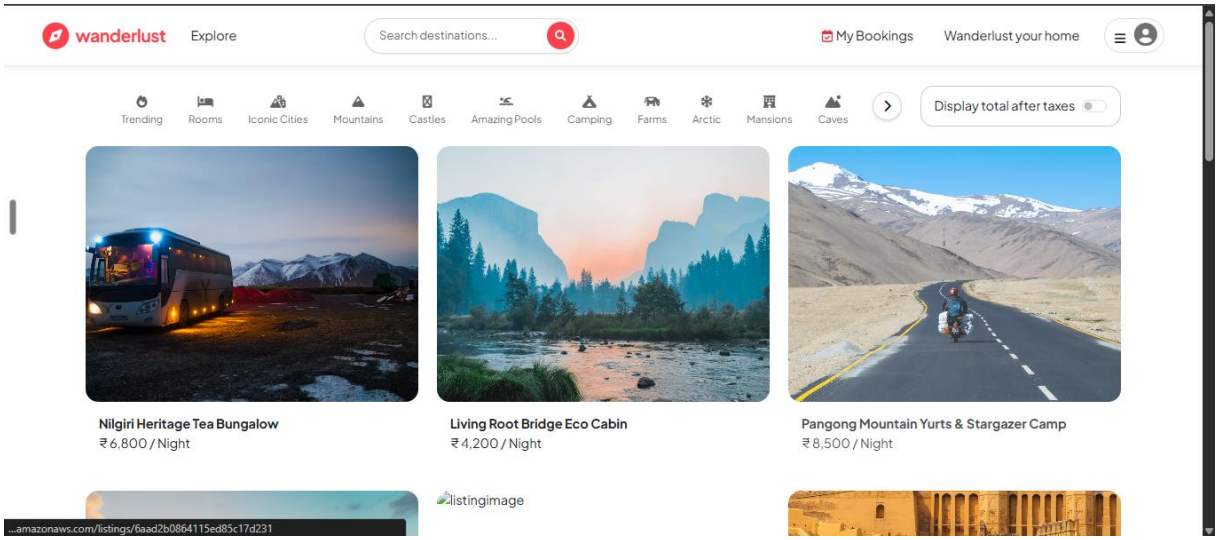


Figure 8.1: Live Wanderlust Application homepage served over AWS Classic Load Balancer with responsive UI across 3 Kubernetes replicas.

8.3 Live Booking & Checkout Flow

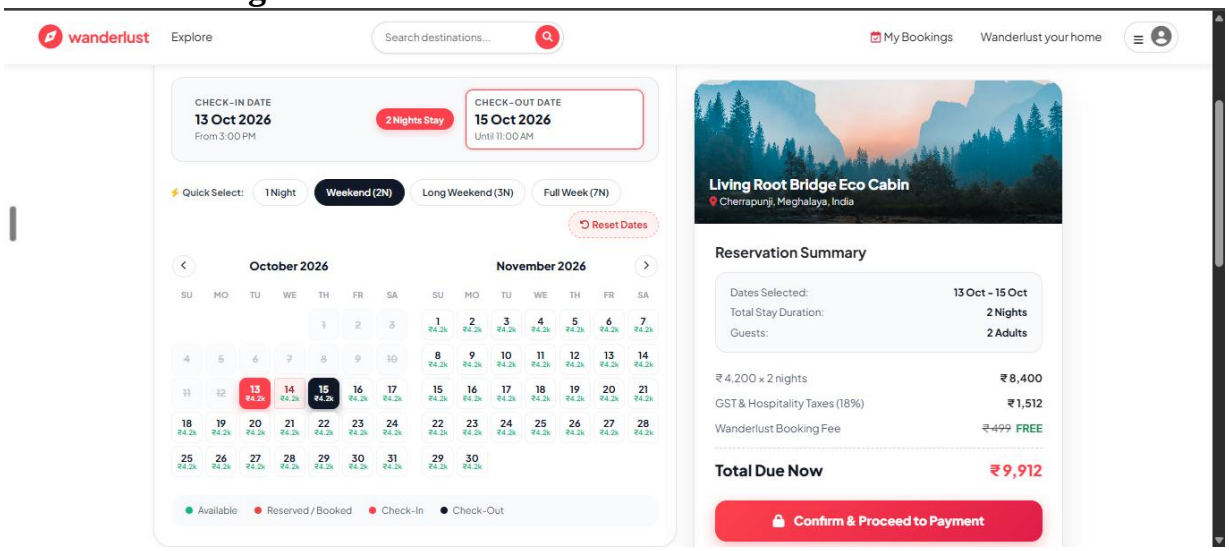


Figure 8.2: Dynamic booking calendar and checkout page confirming session management, cookies, and database state on Kubernetes.

8.4 Pipeline Speed & Deployment Lead Time Results

Table 14: Summary of measured and configured results

Metric	Value
Pipeline stages	5 (checkout, build, push, deploy, post actions)
Verified pipeline run	Build #12, all stages successful
Total pipeline duration	1 minute 35 seconds
Webhook trigger latency	Within approximately 2 seconds of git push
Application replicas	3 (HPA range 3 to 10 at 70% CPU)
Worker nodes	2 x t3.small across ap-south-1a and ap-south-1b
Update strategy	RollingUpdate, maxSurge 1, maxUnavailable 0
Manual steps from commit to production	0

The comparison with the manual process is clear. A manual release involves logging in, pulling code, installing dependencies, restarting the process and checking the site, which typically takes several minutes of attention and carries the risk of human error. The automated pipeline performs an equivalent release in under two minutes, is identical on every run, verifies its own result and can be rolled back with one command.

CHAPTER 9

SECURITY, COST MANAGEMENT & TEARDOWN

9.1 Cloud Security Practices (IAM Least Privilege, Kubernetes Secrets, Helmet CSP)

Security is applied in layers across the infrastructure, the pipeline, the cluster and the application. Table 15 summarises the controls implemented in the project.

Table 15: Security controls by layer

Layer	Control	Purpose
Identity	Separate IAM roles for the EKS cluster and the worker nodes with AWS-managed policies	Least privilege: nodes may only pull from ECR
Pipeline	AWS credentials supplied through Jenkins credential binding; short-lived ECR login tokens	No secrets in source or in the console log
Network	Security group opens only ports 22, 8080 and 80 on the Jenkins host	Limits exposed services
Image	Alpine base, .dockerignore, no embedded credentials	Smaller attack surface and no secret leakage
Cluster	Kubernetes Secrets for the database URL and cookie secret; resource limits per pod	Credentials kept out of images; blast-radius control
Application	Helmet middleware with a Content Security Policy; secure session cookies	Mitigates XSS and common web header risks
Monitoring	Grafana reached by kubectl port-forward rather than a public endpoint	Dashboard is not exposed to the internet

Honest assessment. The project is a demonstration environment, and several settings would be tightened in a real production deployment: restricting the SSH and Jenkins ports to known IP ranges instead of the open internet, running worker nodes in private subnets behind a NAT gateway, running the container as a non-root user, and replacing the default Grafana credentials. These are listed as future work in Chapter 10.

9.2 AWS Cost Optimization & Automated Teardown Strategy (Terraform Destroy)

Managed Kubernetes is the most expensive part of a cloud learning project: the EKS control plane is billed per hour and the Classic Load Balancer is billed while it exists. The project controls cost through several measures:

- **Right-sized compute:** a *t3.micro* for Jenkins and two *t3.small* workers are the smallest practical sizes for this workload.
- **Storage lifecycle:** the ECR lifecycle policy removes untagged images older than 30 days automatically.

- **Pruning:** Docker image pruning on the Jenkins host avoids needing larger disks.
- **Ephemeral environment:** because everything is defined in Terraform, the environment can be created for a demonstration and destroyed afterwards.

The teardown order matters. The Load Balancer and the monitoring stack are created by Kubernetes and Helm rather than by Terraform, so they must be removed first; otherwise the load balancer would remain attached to the VPC and prevent Terraform from deleting the subnets and gateway.

```
# Step 1: Delete the Kubernetes LoadBalancer and the Helm monitoring stack
kubectl delete -f k8s/service.yaml
helm uninstall prometheus-stack -n monitoring
kubectl delete namespace monitoring

# Step 2: Destroy the Terraform-managed infrastructure
cd terraform
terraform destroy -auto-approve
```

After *terraform destroy* completes, no billable resources from the project remain, and the whole environment can be recreated later from the same code.

CHAPTER 10

CONCLUSION & FUTURE SCOPE

10.1 Key Learnings & Project Achievements

This project transformed a conventional web application into a complete cloud-native delivery platform and, in doing so, demonstrated every phase of the DevOps lifecycle with working tools. The principal achievements are:

- A reproducible AWS foundation created entirely by Terraform, covering the VPC, subnets, gateway, Jenkins host, container registry and EKS cluster.
- A fully automated pipeline in which a *git push* results in a verified production rollout in about 1 minute 35 seconds without manual steps.
- A highly available deployment of three pods across two availability zones with rolling updates that never reduce ready capacity.
- Self-healing through liveness probes, automatic pod replacement and elasticity through a Horizontal Pod Autoscaler.
- A professional observability stack that combines the Golden Signals and the USE method in a ten-panel Grafana dashboard.
- A documented cost-control and teardown procedure suitable for a student cloud budget.

Beyond the tools themselves, the main lessons were conceptual. Automation removes a whole class of human error; declarative systems such as Terraform and Kubernetes are easier to reason about than sequences of manual commands; and health checks plus metrics are what turn a running system into an operable one. The project also showed that small design choices, such as ordering Dockerfile instructions for caching or setting maxUnavailable to zero, have large practical effects on speed and reliability.

10.2 Limitations

- Worker nodes and the Jenkins host run in public subnets for simplicity and low cost; a production design would use private subnets and a NAT gateway.
- The pipeline builds, pushes and deploys but does not yet include automated test, security-scan or quality-gate stages before the image is published.
- Kubernetes Secrets are Base64-encoded and not encrypted by default at the manifest level.
- Terraform state is stored locally, which prevents safe collaboration by several engineers.
- A single Jenkins server is a single point of failure for the delivery pipeline.
- The application is exposed over HTTP through a Classic Load Balancer without TLS termination.

10.3 Future Scope (GitOps with ArgoCD, Service Mesh with Istio)

Table 16: Planned enhancements

Enhancement	Description	Benefit
GitOps with ArgoCD	ArgoCD watches the Git repository and reconciles the cluster to the committed manifests, replacing the imperative kubectl set image step	Declarative, auditable and self-correcting deployments with easy rollback through Git
Service Mesh with Istio	Sidecar proxies provide mutual TLS, traffic splitting and telemetry between services	Canary releases, encrypted service traffic and richer request-level metrics
Ingress and TLS	AWS Load Balancer Controller with an ALB Ingress, AWS Certificate Manager and a custom domain	HTTPS, path-based routing and lower cost than a load balancer per service
Secret management	AWS Secrets Manager with the External Secrets Operator, or encrypted secrets at rest	Real secret protection and rotation
Remote Terraform state	S3 backend with DynamoDB state locking	Safe team collaboration and state recovery
Shift-left quality	Add automated tests, image vulnerability scanning (for example Trivy) and linting stages to the pipeline	Defects and vulnerabilities caught before deployment
Hardened containers and network	Non-root container user, private subnets with NAT, network policies, restricted security groups	Reduced attack surface
Alerting	Alertmanager rules for replica count, restarts and node saturation with notifications	Proactive incident response

In conclusion, Wanderlust demonstrates that a modern DevOps toolchain can be assembled, automated and observed by a single engineer, and that the resulting platform delivers the properties that industry expects from production systems: speed, repeatability, resilience and visibility.

APPENDIX & REFERENCES

Appendix A: Kubernetes Command Reference Table

Table 17: Kubernetes and operations command reference

Task	Command	Explanation
View all running pods	`kubectl get pods -o wide`	Shows pods with the node each runs on
View Load Balancer URL	`kubectl get svc`	Shows the external ELB endpoint
Scale pods dynamically	`kubectl scale deployment wanderlust-deployment --replicas=5`	Immediate horizontal scaling
Inspect pod details	`kubectl describe pod <pod-name>`	Image, environment, limits and probe state
Stream live logs	`kubectl logs -l app=wanderlust --tail=50 -f`	Real-time logs from all matching containers
Check rollout history	`kubectl rollout history deployment/wanderlust-deployment`	Lists deployment revisions
Roll back a deployment	`kubectl rollout undo deployment/wanderlust-deployment`	Returns to the previous revision
Connect kubectl to EKS	`aws eks update-kubeconfig --region ap-south-1 --name wanderlust-cluster`	Writes cluster access into kubeconfig
Open Grafana locally	`kubectl port-forward svc/prometheus-stack-grafana 3000:80 -n monitoring`	Access Grafana at localhost:3000

Appendix B: Git Repository & Live Links

Table 18: Project links

Item	Location
GitHub repository	`https://github.com/Swarajbabu/Wanderlust-A-DevOps-Integrated-Travel-Booking-Platform`
Live application (AWS ELB)	`http://a51fe5ad2863047968c94d092fc4c0ce-412112320.ap-south-1.elb.amazonaws.com/listings`
Jenkins pipeline	`http://3.110.81.86:8080/job/wanderlust-pipeline/`
Grafana (port-forward)	`http://localhost:3000`
Demo / viva cheat sheet	`DEMO_CHEATSHEET.md in the repository root`

Note: the live application and Jenkins endpoints are reachable only while the cloud infrastructure is running. They are released when the teardown procedure in Section 9.2 is executed to avoid AWS charges.

References

1 Kubernetes Documentation: Deployments, Services, Probes, Horizontal Pod Autoscaling and Secrets. https://kubernetes.io/docs/

2 HashiCorp Terraform Documentation and the AWS Provider Registry. https://developer.hashicorp.com/terraform/docs

3 Amazon Web Services Documentation: Amazon EKS, Amazon ECR, Amazon VPC and Elastic Load Balancing. https://docs.aws.amazon.com/

4 Jenkins Documentation: Pipeline Syntax and Credentials Binding. https://www.jenkins.io/doc/

5 Docker Documentation: Dockerfile best practices and Docker Compose. https://docs.docker.com/

6 Helm Documentation and the prometheus-community kube-prometheus-stack chart. https://helm.sh/docs/

7 Prometheus and Grafana Documentation. https://prometheus.io/docs/ and https://grafana.com/docs/

8 Beyer, B., Jones, C., Petoff, J. and Murphy, N. R. (eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly Media, 2016 (Chapter 6: Monitoring Distributed Systems, The Four Golden Signals).

9 Gregg, B., *The USE Method*. https://www.brendangregg.com/usemethod.html

10 Kim, G., Humble, J., Debois, P. and Willis, J., *The DevOps Handbook*, IT Revolution Press, 2016.